

# Job Portal System - Full Project Documentation

## 1. Project Overview

The Job Portal System is a full-stack, role-based hiring platform designed as a production-style project with advanced intelligence features for candidate-job matching, recruiter decision support, and admin observability.

### Objectives

- Provide end-to-end hiring workflows for candidate, recruiter, and admin roles.
- Deliver resume-aware recommendations and ATS-like scoring.
- Add explainable and fairness-aware ranking for recruiter shortlisting.
- Support publication-oriented evaluation with baselines, ablation, and fairness metrics.

### Roles

- `CANDIDATE`: profile/resume management, recommendations, applications, eligibility, learning path, interview readiness.
- `RECRUITER`: job posting management, incoming application handling, copilot ranking, trust/safety checks.
- `ADMIN`: users/jobs moderation, application analytics, observability, research evaluation snapshots.

## 2. Technology Stack

- Frontend: React + TypeScript + Vite (`apps/web`)
- Backend: Node.js + Express + TypeScript (`apps/server`)
- Database: PostgreSQL + Prisma ORM
- Authentication: JWT access/refresh tokens with rotation/revocation
- Real-time/Queue: Socket.IO + BullMQ/Redis fallback handling
- Validation/Security: Zod, Helmet, CORS, rate limiting
- Testing: Vitest + Supertest (server)

## 3. Repository Structure

```
```.txt
JobPortalSystem/
  apps/
  server/
  prisma/
  src/
  config/
  constants/
  middlewares/
  modules/
  admin/
  applications/
  auth/
  candidates/
  companies/
  jobs/
  notifications/
  recruiters/
  users/
  observability/
```

queue/  
utils/  
tests/  
web/  
src/  
api/  
styles/  
types/  
App.tsx  
data/  
evaluation-sample.csv  
docs/  
api/openapi.yaml  
FULL\_PROJECT\_DOCUMENTATION.md  
ieee-paper-outline.md  
novelty-claim-and-contributions.md  
experimental-results-template.md  
scripts/  
eval.ts  
docker-compose.yml  
package.json  
````

## 4. System Architecture

### 4.1 High-Level Flow

1. User authenticates and receives access/refresh tokens.
2. Frontend calls REST APIs under `/api/v1/\*` with bearer token.
3. Express routes execute validation, RBAC checks, business logic, and Prisma queries.
4. Notifications are created and optionally pushed asynchronously.
5. Frontend renders role-specific dashboard sections.

### 4.2 Key Runtime Components

- `apps/server/src/app.ts`: middleware stack + route registration.
- `apps/server/src/server.ts`: HTTP server, socket initialization, queue initialization.
- `apps/server/src/config/prisma.ts`: Prisma client singleton.
- `apps/server/src/config/socket.ts`: Socket.IO setup and user room joins.
- `apps/server/src/queue/notification-queue.ts`: BullMQ worker/queue with in-process fallback.
- `apps/server/src/observability/metrics.ts`: in-memory route metrics collector.

## 5. Database Design

The Prisma schema includes:

- Core entities: `User`, `CandidateProfile`, `RecruiterProfile`, `Company`
- Hiring entities: `Job`, `Application`
- Messaging/session entities: `Notification`, `RefreshToken`
- Enums: `Role`, `JobType`, `ExperienceLevel`, `WorkMode`, `JobStatus`, `ApplicationStatus`, `NotificationType`

Notable constraints:

- Unique email for `User`
- One candidate/recruiter profile per user
- Unique application constraint on `(jobId, candidateId)`

- Indexed frequently queried fields (status, role, createdAt, etc.)

## 6. Backend Modules

### 6.1 Auth (`/api/v1/auth`)

Main capabilities:

- Register candidate/recruiter
- Login with JWT token pair
- Refresh token rotation
- Logout and logout-all
- `me` endpoint for current user profile

Security controls:

- Strong password validation via Zod
- Refresh token hashing (`sha256`) in DB
- Revocation map for rotated/logged-out refresh tokens

### 6.2 Candidates (`/api/v1/candidates`)

Main capabilities:

- Candidate profile CRUD (self)
- Resume upload (`multipart/form-data`) and parsing
- Resume analysis and role suggestions
- Skill-gap roadmap generation
- Job eligibility reporting with ATS-style thresholds
- Auto-apply to top eligible jobs
- Adaptive learning path generation
- Interview readiness report
- Candidate application listing

### 6.3 Jobs (`/api/v1/jobs`)

Main capabilities:

- Public open jobs listing with filters
- Recruiter job create/update/delete and status updates
- Candidate recommendations based on hybrid score
- Recruiter copilot ranking with explainability and fairness snapshot
- Job scam-risk/trust analysis
- Recruiter/admin viewing applications per job

### 6.4 Applications (`/api/v1/applications`)

Main capabilities:

- Candidate apply to open job
- ATS score calculation for application
- Recruiter/admin status updates and notes
- Candidate withdrawal
- Notification enqueue on important transitions

### 6.5 Notifications (`/api/v1/notifications`)

Main capabilities:

- List user notifications
- Mark single notification as read
- Mark all unread notifications as read

Note: backend notifications APIs remain active; notification section has been removed from visible frontend output as requested.

## 6.6 Admin (`/api/v1/admin`)

Main capabilities:

- User listing and activation toggle
- Job listing and deletion
- Application aggregate stats
- Observability summary (slow/error-heavy routes)
- Research evaluation endpoint for publication metrics

Research endpoint:

- `GET /api/v1/admin/research/evaluation`
- Returns dataset summary, baseline comparison, ablation, fairness comparison, and operational metrics.

## 7. Frontend Application

### 7.1 Entry and API Client

- Entry: `apps/web/src/main.tsx`
- Root app: `apps/web/src/App.tsx`
- Axios client: `apps/web/src/api/client.ts`

### 7.2 UX Behavior

- Token and user profile persisted in `localStorage`
- Role-based rendering for candidate/recruiter/admin sections
- Candidate flows: resume upload, recommendations, ATS/eligibility, auto-apply, roadmap, adaptive and readiness reports
- Recruiter flows: open jobs, create job, incoming applications, copilot ranking and scam risk
- Admin flows: application stats, observability summary, research evaluation snapshot

### 7.3 Styling

- Main stylesheet: `apps/web/src/styles/index.css`
- Responsive grid/card based layout
- Modal UI for auto-apply confirmation

## 8. AI/Intelligence and Scoring Logic

### 8.1 Resume and Skill Intelligence

Implemented in `apps/server/src/modules/candidates/resume-analysis.ts`:

- PDF resume text extraction (`pdf-parse`)
- Skill extraction via curated keyword map
- Role suggestions from role-skill mapping
- Semantic resume-job score using cosine similarity

### 8.2 ATS and Eligibility

Used in applications/candidates routes:

- Weighted score composition (skills + semantic + experience + profile completeness)
- Eligibility labels (`HIGH`, `MEDIUM`, `LOW`)
- Eligibility threshold parameterization and auto-apply pipeline

### 8.3 Recruiter Copilot

Implemented in `apps/server/src/modules/jobs/job-insights.ts` and jobs routes:

- Explainable candidate score breakdown
- Matched/missing skill reporting
- Fairness snapshot (selection rates + disparate impact)

#### 8.4 Scam-Risk Heuristics

Signals include:

- suspicious text phrases
- missing salary fields
- unrealistic fresher compensation flags
- low-trust/public-email company domain indicators
- duplicate posting signals

#### 8.5 Adaptive Candidate Guidance

Implemented in `apps/server/src/modules/candidates/adaptive-engine.ts`:

- Priority skill derivation from rejection/trend signals
- Weekly learning plans and projected success lift
- Interview readiness MCQ/coding tasks and recommendations

### 9. API Summary (Selected)

#### Public/General

- `GET /api/v1/health`
- `GET /api/v1/jobs`

#### Auth

- `POST /api/v1/auth/register`
- `POST /api/v1/auth/login`
- `POST /api/v1/auth/refresh-token`
- `POST /api/v1/auth/logout`
- `POST /api/v1/auth/logout-all`
- `GET /api/v1/auth/me`

#### Candidate

- `GET /api/v1/candidates/me`
- `PATCH /api/v1/candidates/me`
- `POST /api/v1/candidates/me/resume`
- `GET /api/v1/candidates/me/resume-analysis`
- `GET /api/v1/candidates/me/skill-gap-roadmap`
- `GET /api/v1/candidates/me/job-eligibility`
- `POST /api/v1/candidates/me/apply-top-eligible`
- `GET /api/v1/candidates/me/adaptive-learning-path`
- `GET /api/v1/candidates/me/interview-readiness`
- `GET /api/v1/candidates/me/applications`

#### Jobs

- `GET /api/v1/jobs/recommendations/me`
- `GET /api/v1/jobs/:jobId`
- `POST /api/v1/jobs`
- `PATCH /api/v1/jobs/:jobId`
- `PATCH /api/v1/jobs/:jobId/status`
- `DELETE /api/v1/jobs/:jobId`

- `GET /api/v1/jobs/:jobId/copilot-ranking`
- `GET /api/v1/jobs/:jobId/scam-risk`
- `GET /api/v1/jobs/:jobId/applications`

## Applications

- `POST /api/v1/applications`
- `GET /api/v1/applications/:applicationId`
- `GET /api/v1/applications/:applicationId/ats-score`
- `PATCH /api/v1/applications/:applicationId/status`
- `PATCH /api/v1/applications/:applicationId/notes`
- `DELETE /api/v1/applications/:applicationId`

## Admin

- `GET /api/v1/admin/users`
- `PATCH /api/v1/admin/users/:userId/status`
- `GET /api/v1/admin/jobs`
- `DELETE /api/v1/admin/jobs/:jobId`
- `GET /api/v1/admin/applications/stats`
- `GET /api/v1/admin/observability/summary`
- `GET /api/v1/admin/research/evaluation`

## Notifications

- `GET /api/v1/notifications`
- `PATCH /api/v1/notifications/:id/read`
- `PATCH /api/v1/notifications/read-all`

## 10. Security and Reliability

- JWT auth + RBAC middleware (`requireAuth`, `requireRole`)
- Request body validation (Zod)
- Helmet security headers
- CORS policy via `CLIENT\_ORIGIN`
- API rate limiting under `/api`
- Centralized error handler with Prisma error mapping
- Request IDs and route-level telemetry

## 11. Real-Time and Queue Behavior

- Socket users join `user:<id>` rooms.
- Notification enqueue supports Redis-backed BullMQ worker.
- If Redis is unavailable, fallback writes notification in-process.

## 12. Setup and Run Guide

### 12.1 Prerequisites

- Node.js (18+)
- npm
- Docker (for local Postgres)

### 12.2 Install and Configure

```
```bash
npm install
docker compose up -d
```

```
cp apps/server/.env.example apps/server/.env
cp apps/web/.env.example apps/web/.env
```
```

### 12.3 Database Bootstrap

```
```bash
npm run prisma:generate
npm run prisma:migrate
npm run prisma:seed
```
```

### 12.4 Run

```
```bash
npm run dev
```
```

### 12.5 Verify

```
```bash
npm run build
npm run test
curl -i http://localhost:4000/api/v1/health
```
```

### 12.6 Access

- Web: `http://localhost:5173`
- API: `http://localhost:4000/api/v1`

Seed accounts:

- Admin: `admin@jobportal.com` / `Admin@123`
- Recruiter: `recruiter@jobportal.com` / `Recruiter@123`
- Candidate: `candidate@jobportal.com` / `Candidate@123`

## 13. Research and IEEE Artifacts

### Included Files

- `docs/ieee-paper-outline.md`
- `docs/novelty-claim-and-contributions.md`
- `docs/experimental-results-template.md`
- `scripts/eval.ts`
- `data/evaluation-sample.csv`

### Evaluation Script

```
```bash
npx tsx scripts/eval.ts
```
```

Optional custom dataset path:

```
```bash
npx tsx scripts/eval.ts /absolute/or/relative/path/to/file.csv
```
```

Expected output structure:

- dataset summary
- baseline comparison (`keyword`, `skill`, `hybrid`)

- ablation (`hybridNoSemantic`)
- fairness comparison (disparate impact ratios)

## 14. Testing and CI

- Server tests located under `apps/server/tests`
- Current suite includes:
  - health endpoint test
  - auth validation tests
- CI workflow exists at `.github/workflows/ci.yml`
- Build and tests are intended to run in CI.

## 15. Environment Variables

`apps/server/.env.example`:

- `PORT`
- `NODE\_ENV`
- `DATABASE\_URL`
- `JWT\_ACCESS\_SECRET`
- `JWT\_REFRESH\_SECRET`
- `JWT\_ACCESS\_EXPIRES\_IN`
- `JWT\_REFRESH\_EXPIRES\_IN`
- `REDIS\_URL`
- `CLIENT\_ORIGIN`
- `RATE\_LIMIT\_WINDOW\_MS`
- `RATE\_LIMIT\_MAX`

`apps/web/.env.example`:

- `VITE\_API\_BASE\_URL`
- `VITE\_SOCKET\_URL`

## 16. Known Notes and Limitations

- `docs/api/openapi.yaml` is concise and not fully exhaustive yet.
- Research metrics currently use lightweight heuristics and sample dataset; real publication should use larger anonymized datasets and reproducible split logs.
- Current tests are foundational and should be expanded for edge/security/performance coverage.

## 17. Suggested Next Improvements

- Expand OpenAPI coverage for all routes and schemas.
- Add integration tests for candidate/recruiter/admin critical paths.
- Add database-backed fairness audit snapshots and scheduled reports.
- Add dashboard export for publication-ready tables/figures.
- Introduce role-specific UI pages/components for maintainability beyond monolithic root app.

---

This document is the comprehensive single-file technical reference for the current implementation state of Job Portal System.